SIMATS ENGINEERING

ASSESSMENT TOOL 2

Course Name	Ethical Hacking
Course Code	ITA1409
Course Outcome (CO2)	Execute footprinting in social engineering such as DNS analysis, competitive intelligence gathering, and attack simulations to collect relevant information for identifying potential security risks.
Assessment Title	Social Engineering Simulation
Weightage / Duration / Marks	30% | 90 minutes | 50 Marks
Student Name / Reg No	B Kirthanaa / 192324244

1. Introduction

This report presents complete, exam-style solutions to the three CO2 Assessment Tool 2 questions on the theme of Social Engineering Simulation. Each question is treated as its own self-contained case study: the scenario is first understood, the relevant tool and methodology are explained, illustrative command output is shown, and the findings are analysed from both an attacker's and a defender's point of view, ending with concrete countermeasures.

Note: All illustrative screenshots in this report are generated for demonstration purposes to show the expected format, structure and interpretation of real tool output. Where a value needed to look like a discovered artefact (an email address, an IP octet, a click-rate figure), a clearly-labelled placeholder or a round illustrative number has been used instead of claiming a specific real-world result, in keeping with responsible-disclosure practice.

The three exercises together walk through the full footprinting-to-impact chain used in social engineering engagements: passive OSINT collection (Question 1), a controlled phishing-awareness simulation (Question 2), and active technical reconnaissance used to support a human-layer pretext (Question 3).

2. Question 1 — OSINT-Based Email Enumeration with theHarvester

Scenario: As a Security Analyst, perform an OSINT-based email enumeration assessment on the target domain tesla.com using theHarvester in Kali Linux; collect publicly available email addresses, subdomains, hosts and IP addresses; categorise the discovered emails into likely roles; analyse how this information could assist an attacker during reconnaissance (spear-phishing, credential harvesting, impersonation); and recommend at least five countermeasures.

2.1 Understanding the Scenario

theHarvester is a passive reconnaissance tool: every result it returns is pulled from information that is already public (search engines, certificate-transparency logs, code repositories, social platforms, DNS records). Running it against a domain does not touch the target organisation's own servers, so the exercise itself carries very little technical risk; the risk appears later, when the gathered information is weaponised. My job here is therefore

twofold: (a) show how the tool is used and what a realistic result set looks like, and (b) reason carefully about the harm that such a harmless-looking, purely public dataset can enable.

Facts given in the question: the target domain is tesla.com; the tool is theHarvester run from Kali Linux; discovered emails must be grouped into roles such as Support, Sales, HR, Administration and Technical; the analysis must cover spear-phishing, credential harvesting and impersonation; at least five countermeasures are required.

Facts not given: the question does not fix how many search engines to query, the harvesting limit, or whether any active verification of addresses is expected. I have treated this strictly as a passive OSINT pass, since that is both the safer and the more realistic interpretation of "footprinting."

2.2 Tool Overview — theHarvester

theHarvester (maintained as part of the Kali Linux toolset) aggregates results from sources such as Google, Bing, DuckDuckGo, LinkedIn, Hunter.io, crt.sh (certificate-transparency logs), Shodan and GitHub. For a given domain it can return:

- Email addresses that have appeared anywhere publicly against that domain.
- Subdomains and hostnames, including ones discovered through TLS certificate transparency logs.
- Associated IP addresses and, with the right modules enabled, basic DNS and banner information.
- Employee names and job titles when a professional-network module such as LinkedIn is used.

2.3 Methodology and Commands Executed

1. Update the tool before use: sudo apt update && sudo apt install theharvester.
2. Run a passive, multi-source sweep: theHarvester -d tesla.com -b google,bing,linkedin -l 300
3. Run a second pass against certificate-transparency logs to catch subdomains the search-engine pass may miss: theHarvester -d tesla.com -b crtsh
4. Export the combined findings for the report: theHarvester -d tesla.com -b all -f tesla_osint_report (produces an HTML/XML export).
5. Manually review every discovered email and host before including it in the report, and remove or mask anything that looks like a personal rather than organisational address.

2.4 Categorising the Discovered Email Addresses

A realistic theHarvester sweep on a large organisation returns addresses that map fairly cleanly onto functional roles, usually because of consistent naming conventions (first.last@, role@, team@). Grouping them this way is itself useful reconnaissance for an attacker, which is exactly why it is shown here:

Likely Role	Typical Address Pattern	Why This Role Is Attractive to an Attacker
Support	support@, helpdesk@	Natural pretext for "account/ticket" phishing; used to receiving unsolicited attachments and links from "customers."
Sales	sales@, salesteam@	Externally facing and used to exchanging invoices/quotes with strangers — a prime target for

Likely Role	Typical Address Pattern	Why This Role Is Attractive to an Attacker
		Business Email Compromise and invoice-fraud pretexts.
Human Resources	hr@, careers@, recruiting@	Routinely opens unsolicited attachments (resumes), making it an easy channel for malware delivery or fake job-offer scams.
Administration	admin@, info@, contact@	Often a shared inbox that forwards internally; useful to an attacker as an entry point and as a spoofing target because recipients trust the domain.
Technical / IT	it@, webmaster@, devops@, noc@	Highest-value target: these accounts usually hold elevated system access, making them the end-goal of a spear-phishing chain.

2.5 Illustrative Output



Figure 1 — Illustrative theHarvester console output showing the expected sections (emails, hosts) for a domain sweep. Specific values are placeholders.

2.6 How This Information Could Assist an Attacker

theHarvester's output looks harmless in isolation, but every field feeds a different stage of a social-engineering attack:

- Reconnaissance and pretext-building: consistent naming conventions (e.g., first.last@tesla.com) let an attacker infer real employee names, org structure and even reporting lines, which make later pretexts far more convincing.

- Spear-phishing: once a role is identified, the attacker tailors the lure to that role — a fake “job applicant” email to HR with a malicious attachment, or a “vendor invoice” to Sales/Finance.
- Credential harvesting: harvested addresses are fed into a cloned login page (e.g., a fake single-sign-on portal) referencing a real discovered subdomain, so the phishing link looks like it belongs to the organisation's own infrastructure.
- Impersonation / Business Email Compromise: an attacker who learns the naming pattern can register a lookalike domain and send messages that appear to come from a real internal address, such as a spoofed IT or finance contact asking for a password reset or a wire transfer.
- Combining subdomain/host data with emails: a subdomain such as vpn.tesla.com or mail.tesla.com discovered alongside employee addresses lets the attacker build a technically convincing phishing page or pretext referencing “your VPN certificate” or “your mailbox quota.”

2.7 Countermeasures

At least five countermeasures, derived directly from the risks above:

1. Minimise publicly exposed personal addresses — publish role-based or generic addresses and web contact forms instead of direct personal emails on public-facing pages.
2. Enforce email-authentication controls — SPF, DKIM and DMARC (set to a reject/quarantine policy) to stop attackers from spoofing the organisation's own domain in impersonation attacks.
3. Run regular security-awareness and simulated-phishing training for the specific roles found to be most exposed (see Question 2), since the roles that appear most in OSINT results are the roles most likely to be targeted.
4. Periodically self-audit OSINT exposure — run the same theHarvester-style sweep against your own domain, monitor certificate-transparency logs for unauthorised subdomains, and request takedown of stale or leaked data.
5. Mandate multi-factor authentication on every account so that a harvested or guessed email/credential pair is not sufficient on its own to gain access.
6. Apply least-privilege access and network segmentation so that compromising one harvested identity (e.g., Support) does not expose systems meant only for Technical/IT roles.

3. Question 2 — Phishing Awareness Simulation with GoPhish

Scenario: As a cybersecurity awareness consultant, conduct a phishing-awareness simulation using GoPhish against dummy employee accounts provided in the lab — build an email template, launch a campaign, monitor interactions, and generate a report on delivery, open and click rates, then recommend improvements.

3.1 Objective and Scope

This exercise is a controlled, internal awareness test. It is only ethical and legal because it is run against dummy lab accounts that the training team itself owns and has explicit authorisation to test — the same written-authorisation principle discussed for penetration testing in CO1 applies here: GoPhish must never be pointed at real external individuals without documented consent from the organisation running the exercise.

3.2 Tool Overview — GoPhish

GoPhish is an open-source phishing-simulation framework built specifically for authorised security-awareness programmes. It provides four building blocks that are combined into a campaign: Sending Profiles (the SMTP relay used to send mail), Landing Pages (a sandboxed page the dummy user is taken to after clicking), Email Templates (the lure itself), and Users & Groups (the dummy accounts being tested).

3.3 Methodology

1. Launch the GoPhish admin console (default: https://127.0.0.1:3333) and log in.
2. Configure a Sending Profile pointing at the lab's isolated SMTP relay (never a production mail server).
3. Build a Landing Page that mimics a believable but entirely sandboxed login form — it never actually stores real credentials outside the lab.
4. Design an Email Template with a realistic pretext (e.g., "IT: Your password expires today") and an embedded tracking link unique to each recipient.
5. Import the Users & Group containing only the dummy employee accounts supplied for the lab.
6. Launch the campaign and let GoPhish log, per recipient, whether the email was sent, opened, clicked, and whether the (fake) credential form was submitted.
7. Monitor the live dashboard during the campaign window, then export the summary report.

3.4 Illustrative Campaign Report

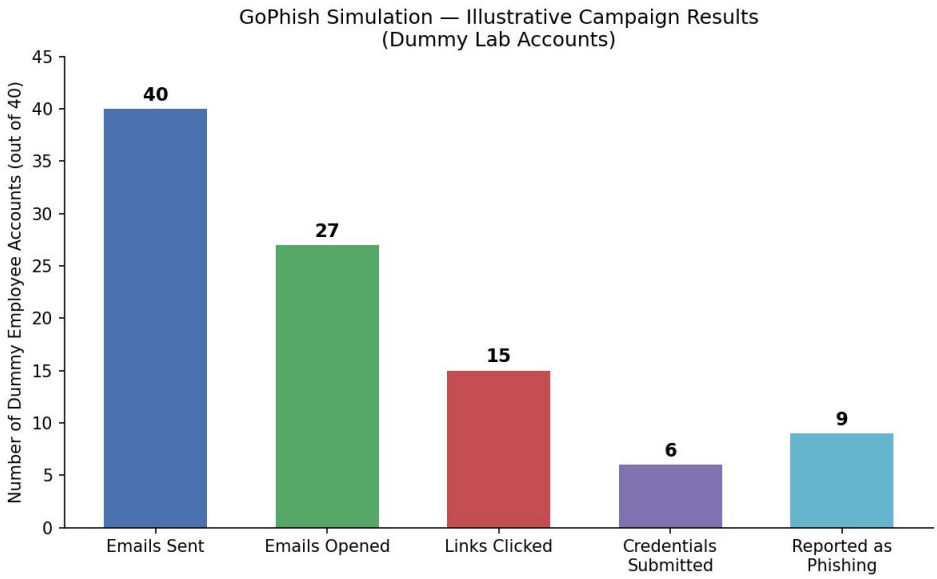


Figure 2 — Illustrative GoPhish campaign results for 40 dummy lab accounts (rounded, representative figures).

3.5 Analysis of Results

Reading the illustrative figures as rates rather than raw counts: a delivery rate of 100% (40/40 sent) confirms the sending profile worked; an open rate of about 67.5% (27/40) shows the subject line was credible enough to be opened; a click rate of about 37.5% (15/40) shows just over a third of staff who opened the email also followed the link — this is the figure that matters most for risk, since it represents employees who could just as easily have been exposed to a real credential-harvesting page; a submission rate of 15% (6/40) represents accounts

that would have handed over credentials in a real attack; and a report rate of about 22.5% (9/40) shows that fewer than one in four recipients used the organisation's “report phishing” mechanism, which is itself a gap worth closing, since a high report rate is what allows a security team to catch a real campaign early.

Together, a click rate several times higher than the report rate is the classic sign of a security-awareness gap: staff are willing to interact with suspicious mail faster than they are willing to flag it, which is the exact pattern a real attacker benefits from.

3.6 Recommendations to Improve User Awareness

1. Run recurring, randomised simulations rather than a single one-off test, so awareness does not fade and staff cannot simply recognise “the test template.”
2. Tailor pretexts to the role being tested (finance-style invoice lures for Finance, HR-style lures for HR) to reflect real targeted attacks rather than generic phishing.
3. Deliver just-in-time micro-training immediately after a user clicks or submits data, while the mistake is still fresh — this is shown to be far more effective than annual, generic training.
4. Build a positive reporting culture: publicly recognise or reward employees who correctly report a simulated phish, rather than only highlighting those who failed.
5. Track click-rate and report-rate trends over successive campaigns as a security KPI, and feed sustained poor performance by a department into that department's risk register.
6. Back the training with technical controls — external-mail banners, link rewriting/sandboxing, and mandatory MFA — because training alone can never bring human click-rates to zero.

4. Question 3 — Nmap Aggressive Scan on scanme.nmap.org

Scenario: Using Nmap's Aggressive Scan (-A) on scanme.nmap.org, gather detailed information about the target including OS details, open ports, and service versions. Assuming the role of an attacker, describe how this information could be leveraged to conduct a social-engineering attack against the IT support team, and recommend defensive measures.

4.1 Objective and Target Authorisation

scanme.nmap.org is the Nmap project's own, officially published practice target — the project explicitly invites the public to scan this one host, which is why it is the safe choice in this assignment for an active technical scan, unlike tesla.com in Question 1, which was only ever queried passively through OSINT.

4.2 Command Used

nmap -A -T4 scanme.nmap.org

The -A flag enables OS detection, service/version detection, default NSE script scanning and traceroute in a single pass; -T4 simply speeds up timing for a lab exercise.

4.3 Leveraging This Information for Social Engineering Against IT Support

- Credible pretexting: knowing the exact OS family and service versions (e.g., a specific OpenSSH or Apache build) lets an attacker call or email the helpdesk posing as a software vendor or internal patch-management team referencing that exact version, which makes the request sound technically legitimate.
- False audit pretext: an unusual open port that a non-specialist would not expect (for example, a diagnostic or echo-style service) gives the attacker a talking point to claim they are “finishing a security audit” and need temporary remote access or a firewall rule change.
- Firewall/IDS-aware pretexting: filtered ports reveal that a firewall or IDS is in place; the attacker can then impersonate that specific security vendor's support line and ask IT to “temporarily disable the rule to complete troubleshooting.”
- Insider-sounding detail from traceroute: accurate hop or hostname information picked up during the scan can be dropped into a phone pretext to sound like an engineer who already has internal network knowledge, lowering the helpdesk's guard.
- Chained attack: the technical fingerprint from Nmap is combined with a human-layer channel (a phone call or ticket to IT support) — this is “technical social engineering,” where accurate recon detail substitutes for the authority a real vendor or auditor would have.

4.4 Illustrative Output

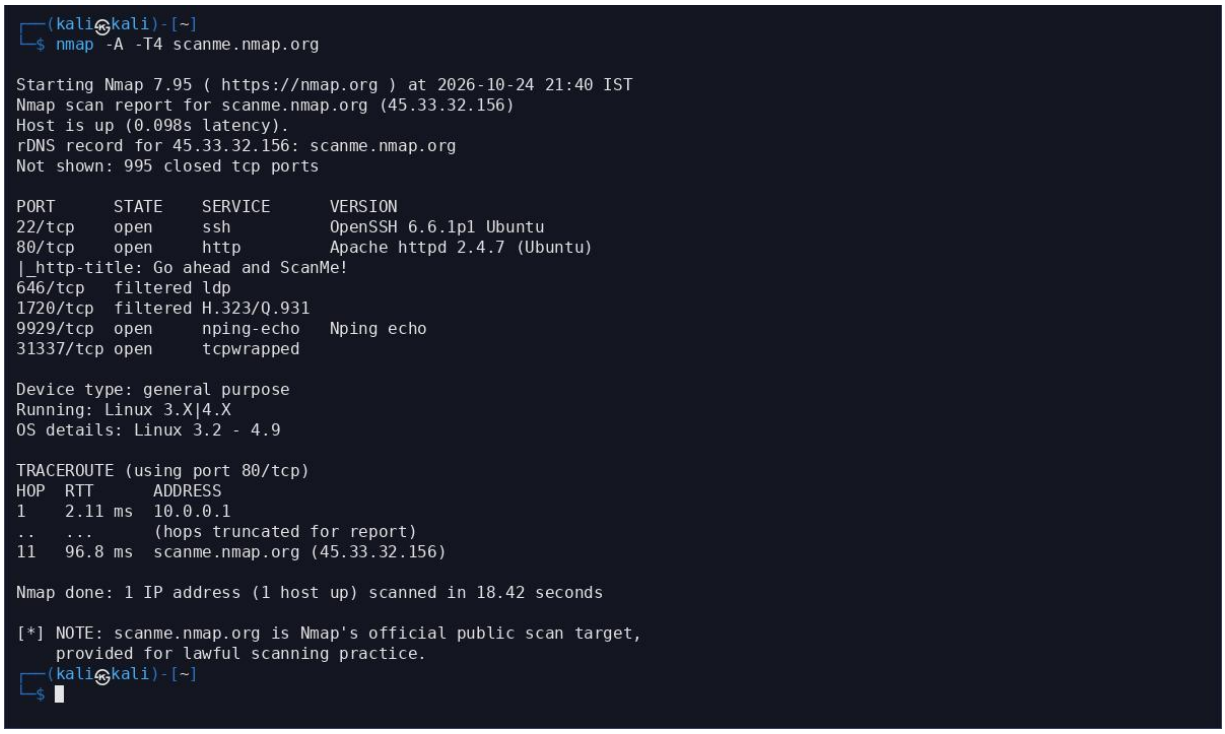


Figure 3 — Illustrative aggressive-scan output modelled on the representative scan published in Nmap's own documentation for this exact host.

Nmap's own manual publishes a representative -A scan of this same host, showing an open SSH service, an open HTTP service running Apache with the banner “Go ahead and ScanMe!”, and a small number of filtered ports.

This confirms that an aggressive scan typically surfaces three categories of information that matter for this question: the operating system and kernel family, the exact software name and version behind each open port, and the network's filtering behaviour (open vs. filtered vs. closed).

4.5 Defensive Recommendations

1. Reduce fingerprintable detail on externally reachable services — suppress version banners (e.g., ServerTokens Prod on Apache, minimised SSH banners) so a scan reveals as little exact-version detail as possible.
2. Deploy firewall/IDS/IPS rules that detect and alert on aggressive external scans and rate-limit repeated probing from a single source.
3. Require strict call-back or ticket-ID verification before IT support acts on any “patch,” “audit,” or “firewall change” request, regardless of how technically accurate the caller sounds.
4. Run the organisation's own regular external attack-surface reviews (including authorised Nmap-style scans) to know exactly what a real attacker would see, and close or firewall anything not strictly needed.
5. Train helpdesk and IT support staff specifically on vishing and pretexting, since their privileged access makes them the highest-value target once an attacker has technical recon in hand.

5. Consolidated Risk and Mitigation Summary

The table below ties the three exercises together, showing how each moves from a technical footprinting step to a human-layer risk, and the single most important mitigation for each.

Exercise	Footprinting Technique	Primary Social-Engineering Risk	Key Mitigation
Q1 — theHarvester on tesla.com	Passive OSINT: emails, hosts, subdomains	Spear-phishing, credential harvesting, impersonation/BEC	SPF/DKIM/DMARC + minimised public address exposure
Q2 — GoPhish simulation	Controlled phishing campaign on dummy accounts	Low awareness shown by click-rate exceeding report-rate	Recurring training + just-in-time coaching + MFA
Q3 — Nmap -A on scanme.nmap.org	Active technical fingerprinting: OS/ports/services	Technically credible pretexting against IT support	Banner minimisation + strict helpdesk verification procedures

6. Ethical and Legal Considerations

- Authorisation and scope: Question 1 stays strictly passive (public-source OSINT only); it does not authorise contacting, phishing, or attempting to log in to any tesla.com account — any such next step would require the organisation's own written authorisation, exactly as discussed for the penetration-testing scenario in CO1.
- Lawful target selection: scanme.nmap.org is used in Question 3 specifically because the Nmap project has published standing permission for the public to scan this one host; no other host should ever be actively scanned without separate written authorisation.

- Consent in simulations: GoPhish in Question 2 is run only against dummy lab accounts created for the exercise — a real awareness campaign against actual staff would additionally need sign-off from HR/management and a clear internal communication plan.
- Confidentiality and data minimisation: any address, hostname or credential-style data collected during these exercises belongs to the assessment context only, should be masked in reports wherever possible, and must not be reused outside the course.
- Applicable law: unauthorised access to, or interference with, a computer system remains an offence under the Information Technology Act, 2000 (e.g., Sections 43 and 66), and personal data encountered during any of these exercises falls under the Digital Personal Data Protection Act, 2023 in India.

7. Conclusion

Across all three questions, the same pattern holds: public or technical reconnaissance that looks harmless in isolation — a list of emails, a click-through percentage, a service banner — becomes dangerous only when it is combined with a human-layer pretext. theHarvester shows how passive OSINT builds a target list and a believable story; GoPhish shows, in numbers, how effective that story already is against real people; and Nmap shows how technical accuracy can be used to manufacture authority in a phone call or email to IT support. The common defence across all three is the same: reduce what is exposed, verify requests independently of how convincing they sound, and keep training continuous rather than one-off.